

GESTURE SENSOR USING ALARM CLOCK

Materials Required:

Esp32

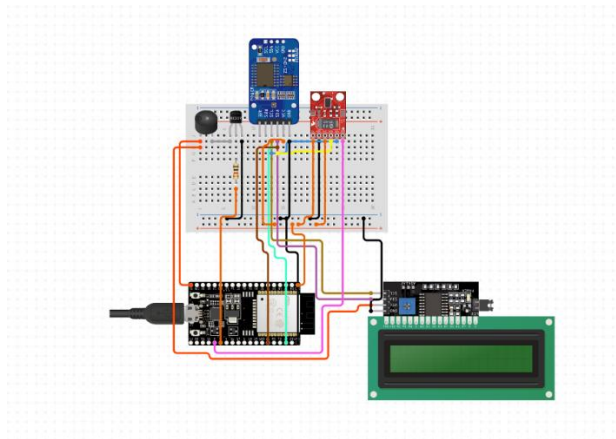
Buzzer 5v

Gesture Sensor – APD9960

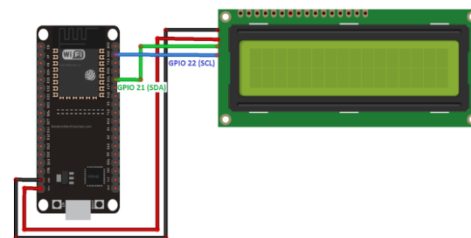
RTC module – DS3231

I2C Display – 16*2

Circuit Connection:

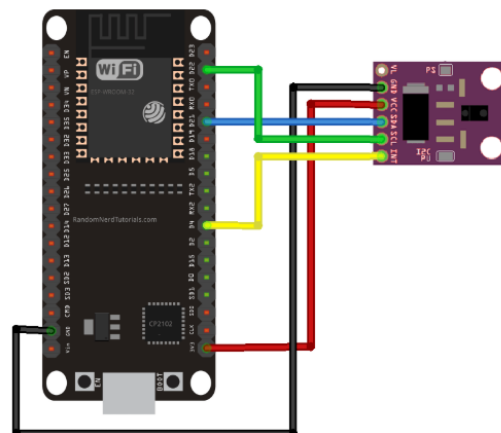


DS3231 RTC Module	ESP32
SQW	GPIO 4 (or any other digital pin)
SCL	GPIO 22
SDA	GPIO 21
VCC	3V3
GND	GND

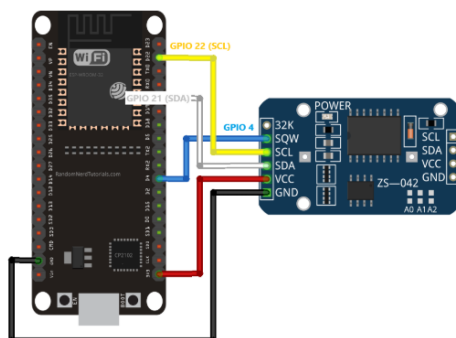


You can also use the following table as a reference.

I2C LCD	ESP32
GND	GND
VCC	VIN
SDA	GPIO 21
SCL	GPIO 22



Pin	Wiring to the ESP32
VCC	3.3V
GND	GND
SDA	GPIO 21
SCL	GPIO 22
INT	GPIO 4 (or any digital pin)
VL	don't connect * (check your jumpers)



GESTURE SENSOR USING ALARM CLOCK

Device	Module Pin	ESP32 Pin / Circuit
DS3231 RTC	VCC	ESP32 3.3V
	GND	ESP32 GND
	SDA	ESP32 GPIO21 (SDA)
	SCL	ESP32 GPIO22 (SCL)
TM1637 display	VCC	ESP32 3.3V
	GND	ESP32 GND
	CLK	ESP32 GPIO23
	DIO	ESP32 GPIO19
APDS9960	VCC	ESP32 3.3V
	GND	ESP32 GND
	SDA	ESP32 GPIO21 (shared I2C)
	SCL	ESP32 GPIO22 (shared I2C)
	INT	(optional) leave unconnected or to a GPIO if using interrupt
TMB12A05 buzzer	+	5V supply
	-	Collector of BC547
BC547 transistor	Collector	Buzzer – (to collector)
	Emitter	GND
	Base	ESP32 GPIO18 via 1 kΩ resistor
Base resistor	1 kΩ	In series between GPIO18 and base
Base pull-down (optional)	10 kΩ	Between base and GND (prevents false ON at boot)
Bluetooth serial	built-in	use BluetoothSerial on ESP32 — no wiring

GESTURE SENSOR USING ALARM CLOCK

Code:

```
#include <Wire.h>
#include <RTCLib.h>
#include <LiquidCrystal_I2C.h>
#include <BluetoothSerial.h>
#include <Adafruit_APDS9960.h>

RTC_DS3231 rtc;
LiquidCrystal_I2C lcd(0x27, 16, 2);
BluetoothSerial SerialBT;
Adafruit_APDS9960 apds;

int buzzerPin = 19;
int alarmHour = -1;
int alarmMinute = -1;
bool alarmSet = false;
bool alarmRinging = false;

// ----- FUNCTION DECLARATIONS -----
void handleBluetoothCommand(String cmd);
void triggerAlarm();
void printTwoDigits(int number);
// ----- SETUP -----

void setup() {
  Wire.begin(21, 22); // I2C pins
  lcd.begin(16, 2);
  lcd.backlight();
  pinMode(buzzerPin, OUTPUT);
  digitalWrite(buzzerPin, LOW);
  Serial.begin(115200);
  SerialBT.begin("ESP32_Alarm");
  Serial.println("Bluetooth Device Ready. Use format: ALARM HH:MM");
  if (!rtc.begin()) {
    Serial.println("Couldn't find RTC");
    lcd.print("RTC Error!");
  }
```

GESTURE SENSOR USING ALARM CLOCK

```
while (1);
}
if (apds.begin()) {
    apds.enableGesture(true);
    Serial.println("APDS9960 Gesture Sensor Ready");
} else {
    Serial.println("APDS9960 not found!");
    lcd.clear();
    lcd.print("No Gesture Sensor");
    delay(2000);
}
lcd.setCursor(0, 0);
lcd.print("Gesture Alarm");
lcd.setCursor(0, 1);
lcd.print("Clock Ready");
delay(1500);
lcd.clear();
}
// ----- LOOP -----

void loop() {
    DateTime now = rtc.now();
    // Display Current Time
    lcd.setCursor(0, 0);
    lcd.print("Time: ");
    printTwoDigits(now.hour());
    lcd.print(":");
    printTwoDigits(now.minute());
    lcd.print(":");
    printTwoDigits(now.second());
    // Display Alarm Status
    lcd.setCursor(0, 1);
    if (alarmSet) {
```

GESTURE SENSOR USING ALARM CLOCK

```
if (alarmRinging) {
    lcd.print("!! ALARM ON !! ");
} else {
    lcd.print("Alarm: ");
    printTwoDigits(alarmHour);
    lcd.print(":");
    printTwoDigits(alarmMinute);
    lcd.print(" ");
}
} else {
    lcd.print("No Alarm Set  ");
}
// Trigger alarm
if (alarmSet && !alarmRinging &&
    now.hour() == alarmHour && now.minute() == alarmMinute) {
    triggerAlarm();
}
// Bluetooth command
if (SerialBT.available()) {
    String input = SerialBT.readStringUntil('\n');
    input.trim();
    handleBluetoothCommand(input);
}
delay(500);
}
// ----- FUNCTIONS -----
void triggerAlarm() {
    alarmRinging = true;
    SerialBT.println("🔔 Alarm Triggered");
    Serial.println("Alarm Triggered");
    lcd.clear();
    lcd.print("ALARM! WAKE UP!");
```

GESTURE SENSOR USING ALARM CLOCK

```
unsigned long startTime = millis();
bool gestureDetected = false;
// Buzzer ON until gesture or 30 seconds timeout
while (millis() - startTime < 30000) {
    // Buzzer sound pulse
    digitalWrite(buzzerPin, HIGH);
    // Check for gesture
    if (apds.gestureValid()) {
        int gesture = apds.readGesture();
        if (gesture == APDS9960_LEFT || gesture == APDS9960_RIGHT ||
            gesture == APDS9960_UP || gesture == APDS9960_DOWN) {
            gestureDetected = true;
            break;}}
    // Stop alarm lcd.clear();
    if (gestureDetected) {
        digitalWrite(buzzerPin, LOW);
        lcd.print("Gesture Stop ");
        SerialBT.println("🚫 Alarm stopped by gesture");
        Serial.println("Alarm stopped by gesture");
    } else {
        lcd.print("Auto Stop 🕒");
        SerialBT.println("Alarm auto stopped");
        Serial.println("Alarm auto stopped (timeout)");
    }
    delay(2000);
    lcd.clear();
    alarmRinging = false;
    alarmSet = false;}

void handleBluetoothCommand(String cmd) {
    cmd.toUpperCase();
    if (cmd.startsWith("ALARM")) {
```

GESTURE SENSOR USING ALARM CLOCK

```
int spaceIndex = cmd.indexOf(' ');
if (spaceIndex != -1) {
    String timePart = cmd.substring(spaceIndex + 1);
    int colonIndex = timePart.indexOf(':');
    if (colonIndex != -1) {
        alarmHour = timePart.substring(0, colonIndex).toInt();
        alarmMinute = timePart.substring(colonIndex + 1).toInt();
        alarmSet = true;
        SerialBT.println("Alarm set to " + String(alarmHour) + ":" + String(alarmMinute));
        Serial.println("Alarm set successfully");
    } else {
        SerialBT.println("Invalid format. Use ALARM HH:MM");
    }
}
} else if (cmd == "CLEAR") {
    alarmSet = false;
    alarmRinging = false;
    digitalWrite(buzzerPin, LOW);
    SerialBT.println("Alarm cleared");
} else {
    SerialBT.println("Unknown command. Use ALARM HH:MM or CLEAR");
}
}

void printTwoDigits(int number) {
    if (number < 10) lcd.print("0");
    lcd.print(number);
}
```

GESTURE SENSOR USING ALARM CLOCK

OUTPUT:

